

AI in signal and audio processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction & Fundamentals of Signal/Audio
- **Session 2:** Audio Features & Librosa
- **Session 3:** Audio Processing with Torchaudio
- **Session 4:** Sound Detection and Classification
- **Session 5:** Sequence Modeling with RNNs and LSTMs
- **Session 6:** Automatic Speech Recognition (ASR)
- **Session 7:** Speaker and Emotion Recognition
- **Session 8:** Project presentation

Audio Processing with Torchaudio

Session 3

Audio Processing with Torchaudio

Session 3

Objectives:

- Understand what Torchaudio is and how it integrates with PyTorch.
- Load, visualize, and transform audio data using Torchaudio.
- Compare Torchaudio with Librosa for audio processing.
- Build a preprocessing pipeline suitable for ML tasks.
- Prepare audio features for classification.

What is Torchaudio?

Torchaudio is a PyTorch domain library that provides audio processing tools for deep learning applications.

Why it matters:

- Seamless integration with the PyTorch ecosystem
- GPU acceleration through Torch tensors
- Optimized for batch processing with Dataloaders
- Includes pre-trained models, datasets, transforms, and I/O utilities

Installation:

```
pip install torchaudio
```

Torchaudio vs librosa

Torchaudio for model pipelines, Librosa for feature engineering

- **Torchaudio:**
 - Tensor-based, GPU-ready
 - Ideal for ML and DL pipelines
 - Modular and differentiable transformations
 - Works well with PyTorch's DataLoader, Dataset, and autograd
- **Librosa:**
 - NumPy-based, CPU-only
 - Rich feature extraction, ideal for analysis and prototyping
 - Excellent visualization and educational tool

Loading Audio with Torchaudio

Torchaudio loads audio as PyTorch tensors

Python:

```
import torchaudio  
waveform, sample_rate = torchaudio.load("audio.wav")
```

Where:

- waveform: Tensor of shape [channels, time]
- sample_rate: sampling rate of the audio

Visualizing Waveform and Spectrogram

Torchaudio loads audio as PyTorch tensors

Python:

```
import torchaudio.transforms as T
import matplotlib.pyplot as plt
```

Waveform

```
plt.figure(figsize=(10, 2))
plt.plot(waveform.t().numpy())
plt.title("Waveform")
plt.show()
```

Spectrogram

```
spec = T.Spectrogram()(waveform)
plt.imshow(spec.log2()[0,:,:].numpy(), cmap='viridis', aspect='auto')
plt.title("Spectrogram")
plt.colorbar()
plt.show()
```


Basic Transforms in Torchaudio

Use prebuilt classes in **torchaudio.transforms**:

- `Resample(orig_freq, new_freq)`
- `Spectrogram(n_fft=400, win_length=None, hop_length=None)`
- `MelSpectrogram(sample_rate=16000, n_fft=1024, hop_length=512)`
- `MFCC(n_mfcc=13)`
- `AmplitudeToDB()`
- Padding and Trimming: ensure consistent audio lengths
 - Padding: Fixes all audio to same duration

Basic Transforms in Torchaudio

- **Resample – Change sampling rate**

```
resampler = T.Resample(orig_freq=48000, new_freq=16000)  
resampled_waveform = resampler(waveform)
```

- **Spectrogram – Linear-frequency spectrogram**

```
spec = T.Spectrogram(n_fft=400, hop_length=160)(waveform)
```

- **MelSpectrogram – Perceptual frequency scale**

```
mel_spec = T.MelSpectrogram(sample_rate=16000, n_fft=1024, hop_length=512)(waveform)
```

- **MFCC – Mel Frequency Cepstral Coefficients**

```
mfcc = T.MFCC(sample_rate=16000, n_mfcc=13)(waveform)
```

- **AmplitudeToDB – Convert to decibel scale**

```
db_spec = T.AmplitudeToDB()(mel_spec)
```

- **Padding – Ensure the same length**

```
lengths = [waveform.shape[1] for waveform in dataset] #choose percentile  
target_len = int(np.percentile(lengths, 95))  
pad_amt = target_len - waveform.shape[1]  
padded = F.pad(waveform, (0, pad_amt))
```

Data augmentation: adding noise

Adding noise is a simple but effective data augmentation technique to simulate real-world conditions:

Why add noise?

- Increases generalization
- Prevents overfitting
- Simulates noisy environments (street sounds, wind, crowds)

Example:

- **White Noise Addition:**

```
def add_white_noise(waveform, noise_level=0.005):  
    noise = torch.randn_like(waveform) * noise_level  
    return waveform + noise
```

Data augmentation: time/frequency masking

Improve model generalization by hiding parts of the spectrogram during training.

Masking Type	Description	Purpose
Time Masking	Randomly masks a range of time frames (horizontal bands)	Simulates dropouts or silent periods
Freq Masking	Randomly masks frequency bins (vertical bands)	Simulates channel loss or occlusion

```
T.TimeMasking(time_mask_param=30),  
T.FrequencyMasking(freq_mask_param=15)
```

Where:

- `time_mask_param`: maximum width (in time frames) to mask
- `freq_mask_param`: maximum width (in frequency bins) to mask

Data augmentation: add effect

Torchaudio.sox_effects.apply_effects_tensor lets you apply SOX (Sound eXchange, the Swiss Army knife of audio manipulation) audio effects programmatically to a waveform. This is ideal for adding realism or creating synthetic variations in data.

```
torchaudio.sox_effects.apply_effects_tensor(waveform, sample_rate, effects_list)
```

Example:

```
effects = [  
    ["lowpass", "-1", "300"],    # Apply single-pole lowpass filter  
    ["speed", "0.8"],           # Reduce playback speed  
    ["rate", f"{sample_rate}"], # Adjust sample rate back to original  
    ["reverb", "-w"]             # Add wet reverb  
]
```

This is useful for:

- Simulating different microphones/environments
- Building data augmentation pipelines
- Testing robustness of models

Data augmentation: add effect

Effect	Description	Key Parameters	Example Usage
Reverb	Adds reverberation (echo-like effect). Can simulate small to large spaces.	-w (wet-only), reverberance, HF-damping, room-scale, stereo-depth, pre-delay, wet-gain	["reverb", "-w", "50", "50", "80"] → Wet-only, room size 80%
		Default values: 50 50 100 100 0 0 (if not specified)	["reverb"] → basic reverb with default dry/wet mix
speed	Changes the speed and pitch together.	Factor (e.g., 1.2 = +20%, 0.8 = -20%)	["speed", "0.8"] → slower and lower pitch
tempo	Changes the speed without changing pitch.	Factor (>1 = faster, <1 = slower)	["tempo", "1.1"] → 10% faster without pitch shift
pitch	Shifts pitch without changing speed.	Value in cents (100 = 1 semitone)	["pitch", "200"] → 2 semitones higher
lowpass	Applies a low-pass filter (keeps frequencies < cutoff)	Cutoff frequency in Hz	["lowpass", "300"] → suppress high frequencies
highpass	Applies a high-pass filter (keeps frequencies > cutoff)	Cutoff frequency in Hz	["highpass", "1000"] → suppress low frequencies
equalizer	Adjust specific frequency bands	Frequency, width (Q), gain	["equalizer", "1000", "2.0q", "-10"]
norm	Normalize audio to target amplitude level	Target gain in dB or -n for RMS normalization	["norm", "-3"] → normalize to -3 dB
compond	Apply compression/expansion to dynamic range	Attack/decay, transfer function	["compond", "0.3,1", "-60,-20,-10", "-5", "-90", "0.2"]
vol	Adjust volume	Gain in dB or linear scale	["vol", "0.5"] → half the volume
rate	Resample audio to a new sample rate	New sample rate	["rate", "16000"] → resample to 16kHz

Data augmentation: time shifting

What is Time Shifting?

You roll (shift) the audio waveform left or right along the time axis, without changing its content.

Why use it?

- Increases variability of training data
- Simulates timing variations in speech or sound events
- Helps models become invariant to when a sound occurs

```
def time_shift(waveform, shift_limit):  
    shift = int(np.random.uniform(-shift_limit, shift_limit))  
    return torch.roll(waveform, shifts=shift, dims=1)
```

Audio Preprocessing Pipeline

Create a full audio pipeline using `torch.nn.Sequential`:

```
audio_transform = torch.nn.Sequential(  
    T.Resample(orig_freq=48000, new_freq=16000), # Resample to 16kHz  
    T.MelSpectrogram(sample_rate=16000,          # Compute Mel spectrogram  
                      n_fft=1024,  
                      hop_length=512,  
                      n_mels=64),  
    T.AmplitudeToDB(),                          # Convert to decibels  
    T.TimeMasking(time_mask_param=30),          # Apply time masking  
    T.FrequencyMasking(freq_mask_param=13)      # Apply frequency masking  
)
```


Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):  
    def __init__(self, file_list):  
        self.files = file_list  
        self.transform = audio_transform # use Sequential transform defined previous slide  
  
    def __getitem__(self, idx):  
        waveform, sr = torchaudio.load(self.files[idx])  
        return self.transform(waveform), self.files[idx] # optional: return filename  
  
    def __len__(self):  
        return len(self.files)  
  
from torch.utils.data import DataLoader  
loader = DataLoader(AudioDataset(file_list), batch_size=16, shuffle=True)
```